



SAT solveri

Zerina Pandža, Enis Adilović, Faris Aljić, Nedžmija Karić, Fadila Begić

April 2024

Contents

1	Uvod	2
1.1	Šta je SAT problem?	2
1.2	SAT i NP kompletnost	3
1.2.1	Početak i zasluge	3
1.2.2	NP kompletni problemi	3
2	Razvoj kroz historiju	6
3	Vrste SAT solvera	11
3.1	DPPL-algoritam	12
3.2	CDCL-algoritam	13
3.3	Paralelni SAT-solveri	14
3.4	Portfolio	14
3.5	Zavadi pa vladaj	14
3.6	Lokalno pretraživanje	14
3.7	Slučajni pristup (Randomized approaches)	15
4	Boolean operatori	16
5	Zadovoljivost Booleove logike i SAT solveri	18
6	Tseitinova transformacija	20
7	SAT konstrukcije	20
7.0.1	Jednakost (Same)	21
7.0.2	Tačno jedan (Exactly one)	21
7.0.3	Bar K, Manje od K, Tačno K (At least K, Less than K, Exactly K)	21
8	Bojenje grafika	23
9	Primjena	25
9.1	Primjer 1	26
9.2	Primjer 2	28
9.3	Primjer 3	31
10	Zaključak	34
11	Izvještaj	34
12	Izvori	35

1 Uvod

1.1 Šta je SAT problem?

Boolean Satisfiability Problem, ponekad nazvan Propositional Satisfiability Problem - skraćeno SAT, SATISFIABILITY ili B-SAT - bi se mogao prevesti na naš jezik kao problem zadovoljivosti. Radi lakšeg čitanja, nadalje će se koristiti termin SAT. Prije nego što objasnimo šta je to SAT, moramo se upoznati sa Booleovim formulama.

Koristimo se Booleovom logikom u kojoj pojednostavljamo događaje da budu ili 0 ili 1, tačno ili netačno. Booleova formula je konačni niz koji se sastoji od: varijabli, Booleovih operatora (negacija, disjunkcija, konjunkcija, implikacija, ekvivalencija) te zagrada(,). Dakle, Booleova formula uzimamo dati izraz i zapisujemo ga kao varijable spojene putem spomenutih operatora.

SAT (eng. Boolean satisfiability problem) je problem odlučivanja da li je neka Booleova formula zadovoljavajuća (eng. satisfiable) ili nije. On je paradigmatički problem zadovoljivosti ograničenja sa brojnim primjenama, uključujući dizajn hardvera i softvera, operacijska istraživanja, bioinformatiku i još mnogo toga. Konceptualno, SAT nije težak problem za riješiti. Svaki atom u formuli odgovara binarnom izboru, i s njima se može postupati konačan broj puta.

Iako naizgled jednostavan, SAT spada u grupu NP kompletnih (NP complete) problema za koje dosad nije pronađen algoritam čiji je period izvršavanja efikasniji od eksponencijalnog, tj. algoritam koji se ne oslanja na brute-force isprobavanje mogućnosti dok se odgovor ne pojavi. Štaviše, kompleksnost tih algoritama toliko raste da oni postaju neupotrebljivi. Teži se ka tome da se porast potrebnog vremena rješavanja svede sa eksponencijalnog na polinomijalni, što bi otvorilo mnoga vrata i mnogostruko poboljšalo efikasnost postojećih sistema.

Postoje mnogi poznati NP kompletni problemi, koji opet predstavljaju samo djelić njihovog stvarnog broja. Rješavanje jednog od ovih problema riješilo bi ih sve - jer svi dijele osobinu da nijedan algoritam brži od eksponencijalnog nije pronađen da ih riješi. Pronalaženje polinomskog algoritma vremena za jedan od njih ustupilo bi mjesto pronalaženju polinomskog algoritma vremena za sve njih.

“SAT je kamen temeljac teorije računske kompleksnosti (computational complexity theory), a postao je centralni problem za rješavanja teških računskih problema u praksi. Od revolucije u SAT rješavanju problema odlučivanja koja se dogodila oko milenijuma, postignuta su značajna poboljšanja efikasnosti, a dodane su i nove metode za provjeru i povjerenje. U posljednjih 10 godina, SAT je dalje evoluirao širenjem svojih primjena, uključujući optimizaciju, brojanje, pa čak i probleme koji uključuju kvantifikatore.”

1.2 SAT i NP kompletnost

1.2.1 Početak i zasluge

An algorithm is a finite answer to an infinite number of questions

- Stephen Kleene

Pojam NP kompletnosti nije postojao do sedamdesetih godina, a zaslužno je više matematičara, što može stvoriti zabunu oko egzaktnih zasluga. Renato Lui Geh u svom radu ovako navodi slijed događaja: "Američki matematičar Stephen Cook objavio je 1971. godine "Složenost procedura dokazivanja teorema" ("The Complexity of Theorem-proving Procedures"), a sljedeće godine je njegov sunarodnik Richard Karp objavio "Svodljivost među kombinatornim problemima" ("Reducibility Among Combinatorial Problems") . Dva članka predstavila su koncepte P i NP klasa, što je dvojcu donijelo Turingovu nagradu. Zanimljivo je da je 1973. godine, s druge strane Gvozdene zavjese, Ukrajinac Leonard Levin objavio u SSSR-u ekvivalentne rezultate Cookovim i Karpovim, ali uzimajući u obzir probleme pretrage umesto problema odlučivanja (zanimljiva je napomena da Levin nije dobio Turingovu nagradu za svoj rad, uprkos postizanju ekvivalentnih rezultata).

Ovaj nezavisni, paralelni rad iz suprotnih dijelova svijeta, ideološki i geografski, doveo je do onoga što se smatra jednim od najvažnijih otvorenih pitanja u teorijskoj informatici: P vs NP problem."

1.2.2 NP kompletni problemi

Za praktične svrhe razlika između polinoma i eksponencijalnog sistema je često važnija od razlike između konačnog i nekonačnog

- Jack Edmonds, 1965

SAT je prvi NP kompletni problem, jer se sam pojam NP kompletnosti vezuje za otkriće ovog svojstva kod SAT problema. Dosad smo rekli šta je SAT i da spada u klasu NP kompletnih problema, te je red da pojasnimo šta tačno znači ovaj termin. (Ovo objašnjenje izvedeno je uglavnom iz literature pod (2), no oslanja se i na neke manje dopune iz (1), pojednostavljeno). Teorija računske složenosti razvrstava probleme u klase na osnovu toga koji tipovi algoritama ih mogu riješiti i koliko im je resursa potrebno, tj, koji tipovi algoritama se mogu upotrijebiti za provjeru dobivenog rješenja. Algoritmi se dijele na eksponencijalne i polinomijalne, ovisno o tome kakva je stopa vremenskog rasta njihove izvedbe. Imajući ovo u vidu, dobivamo tri klase: eksponencijalno vrijeme rješavanja i provjere (EXP), polinomijalno vrijeme rješavanja i provjere (P) i eksponencijalno vrijeme rješavanja i polinomijalno vrijeme provjere (NP). Zaista, ma koliko teško bilo pronaći efikasan algoritam rješavanja NP problema, provjera je uvijek daleko jednostavnija.

Ovo je samo dio slagalice. Zbog čega N u NP? N se odnosi na nedeterminizam (non-deterministic polynomial). Krenimo od objašnjavanja determinizma i determinističkog računanja (deterministic computation). To je vrsta koja nam je poznata - u vremenu se dešava jedna stvar i postoji samo jedan mogući sljedeći korak u zavisnosti od toga gdje se nalazimo u računanju. Algoritam radi jednu po jednu stvar i uvijek je određen sljedeći korak. Ovo nije slučaj sa nedeterminističkim računanjem. Postoje dva načina razmišljanja o tome: ili nedeterministički algoritam magično pogađa rješenje ili se grana na sve moguće izbore i provjerava sve grupe paralelno u istom vremenskom periodu kao i deterministički. Provjera svih putanja paralelno je računski ekvivalentna pogađanju pravog rješenja iz prvog pokušaja. Ovo očigledno nije realan ili praktičan model računanja. Nedeterministički model izračunavanja nije napravljen da bi modelirao realno računanje, već je više alat za razumijevanje problema, za njihovo kategoriziranje. On objedinjuje sve ove NP probleme i pokazuje da su u suštini svi isti. Ako možete riješiti jedan, onda ste riješili ovaj apstraktni koncept pogađanja, zbog čega ste ih riješili sve. Dakle, NP Kompletni problemi mogu riješiti stvari u polinomskom vremenu ako imaju nedeterministički algoritam. Teško ih je riješiti, ali ih je lako provjeriti.

Potpunost - zašto rješavanje jednog rješava sve? Tokom 70-ih naučnici su radili na ovim problemima u izolaciji, sve dok Steven Cook i Leonard Levin nisu otkrili njihovu vezu. Slično kao što je Descartes povezao algebru i geometriju, Cook i Levin su pokazali vezu između poznatog NP problema zvanog SAT i računanja. Oni su pokazali da SAT problem može izvršiti računanje. Matematički problem koji vrši račun? Baš kako ljudski jezik ima moć opisivanja objekata u svijetu, tako i Bulove formule imaju sličnu moć s matematičkim konceptima. Često daju odgovore na pitanje "Da li objekat X sa svojom Y postoji?" Cook i Levin su pokazali kako SAT formule mogu odražavati ponašanje nedeterminističke Turingove mašine.

Turingova mašina može raditi sve što može i moderni računar, i po prirodi je deterministična. Ima jednu traku sa nulama i jedinicama i glavu koja se kreće po traci i može brisati i pisati po njoj, ovisno o instrukcijama koje su joj date. Nedeterministička Turingova mašina ima više traka, može se razgranati i slijediti nekoliko uputa, ne mora biti samo jedna po jedna. Sve dok je jedna od opcija ispravna, NDTM će je pronaći, tako da može pokrenuti bilo koji ND algoritam i riješiti bilo koji NP problem. Cook-Levin teorem - SAT formula može predstavljati bilo koju NDTM, tako da može odgovoriti na bilo koje pitanje koje NDTM može. Pokazivanje toga je jako dugo sa puno složene matematike - cijelo predavanje, a možda i duže.

Kada bismo mogli riješiti opći SAT problem, mogli bismo simulirati NDTM i time riješiti sve ostale NP probleme. U svom radu, Karp je pokazao da je još 21 problem bio NP kompletni na isti način kao i SAT. Oni također mogu simulirati NDTM. Postoje hiljade NP potpunih problema i teorija kompletnosti NP-a pokazuje da je njihova osnovna poteškoća dio sa pogađanjem, ali bi ga rješavanje za jedan problem riješilo i za sve ostale. Naučnici zapravo ne misle da ćemo ikada riješiti NP probleme. To bi značilo da nedeterminizam nije bio moćniji od determinizma i da je provjeriti da li je nešto ispravno jednako lako

kao i pronaći rješenje za to. Ali niko nije dokazao da rješenje ne postoji, pa je pitanje da li je $P=NP$ najveće otvoreno pitanje informatičke nauke.

Doprinos Cook Levin teoreme je da sada da biste pokazali da je problem NP comp samo trebate uzeti poznati NP kompletan problem i svesti ga na vaš novi problem. Mnogo lakše nego ga smanjiti iz NDTM. Danas, kada je suočen s novim teškim problemom, često je prvi instinkt CS-a da pokuša svesti poznati NP potpuni problem na novi problem. Štedi mnogo vremena, umjesto da pokušavate to riješiti. Problem A se svodi na problem B ako možemo koristiti rješenje iz problema B da riješimo A. Rješenje B ubija dvije muhe jednim udarcem, našli smo rješenje za A, što znači da A nije teže od B.

2 Razvoj kroz historiju

Kako smo uočili nevjerovatan napredak u računarskom hardveru, koji daje sve brže procesore, veliku i efikasnu memoriju i masivno paralelne računarske jedinice, moglo bi se zapitati da li je napredak u rješavanju SAT-a samo rezultat napretka hardvera. Upravo se Time Leap Challenge bavio ovim pitanjem trkom između 20-godišnjih SAT solvera na novom računarskom hardveru i modernih SAT solvera na 20-godišnjem kompjuterskom hardveru. Iako hardverska poboljšanja čine stare solvere bržima, algoritamski napredak dominira i pokreće današnje SAT solvere. Ma kako dobar hardver bio, ne uspijeva nadmašiti bolje napisan algoritam.

SAT solver se mogu klasificirati u dva tipa.

Kompletni algoritmi garantuju da će vratiti SAT ili UNSAT (iako im može trebati nepraktično dugo vremena da to urade).

Nepotpuni algoritmi vraćaju SAT ili vraćaju UNKNOWN (tj. vraćaju se bez davanja odgovora).

Ovo su bili najčešći SAT solveri 1990-ih. Nepotpuni solveri, zasnovani na stohastičkom (slučajnom) lokalnom pretraživanju, uspješno su primijenjeni na probleme planiranja i zadovoljavajuće instance. Nasuprot tome, potpuni solveri, bazirani na traženju povratnog traga, korišteni su za rješavanje kombinatornih problema, kao i uniformne slučajne k-SAT formule (uključujući one koje se ne mogu zadovoljiti). Ovi rani potpuni solveri slijedili su opći pristup, nazvan DPLL, koji su prvi predložili Davis, Logemann i Loveland (DLL) kao memorijsko efikasnu rafinaciju ranijeg algoritma od strane Davisa i Putnama (DP). DPLL kombinuje DP i DLL da naglasi oba doprinosa.

Prvi stub revolucije bio je solver GRASP, koji je predložio novu arhitekturu, kombinujući nehnološko vraćanje unazad, analizu konflikta i učenje, koje se danas naziva učenje vođeno konfliktom (CDCL - conflict driven clause learning). CDCL je više od običnog DPLL-a sa učenjem, jer više nije čista pretraga unazad. Moderni SAT solveri koriste strukturu podataka traga (trail) koju je predložio MiniSat za snimanje stabla pretraživanja i grafa implikacije. Drugi stub bio je solver Chaff baziran na CDCL-u, posebno dizajniran za rješavanje velikih instanci referentnih vrijednosti i postizanje neviđene ravnoteže između sofisticiranosti algoritama i struktura podataka s jedne strane i praktične efikasnosti s druge strane. Chaff je predstavio strukturu podataka Watched Literal, "lijenu" šemu za izvođenje jedinične propagacije koja je omogućavala besplatno vraćanje unazad. Heuristika grananja (VSIDS) i procedura analize konflikta pažljivo su dizajnirani sa novim kompromisom između snage rezonovanja i troškova računanja: naglasak na nedavnim sukobima vodi pretraživanju zasnovanom na lokalitetu.

Za memoriju je prilično zahtjevna činjenica da tokom pretrage dođe do pamćenja mnogih konfliktnih klauzula. Moderni CDCL solveri često brišu naučene klauzule i heuristički predviđaju one koje treba zadržati. Različite heuristike, kao što su VSIDS, EVSIDS ili LRB, dostupne su za optimizaciju pretrage i dodeljivanje varijabli. Te heuristike nameću niske troškove, ne stvaraju gotovo nikakve troškove za vraćanje unazad i ažuriraju se prilikom učenja klauzula.

Da bi izbjegli nesretne prolaskе kroz stablo pretraživanja tokom rješavanja, solveri često poništavaju dodjelu svih varijabli i ponovo pokreću pretragu (ali zadržavaju naučene klauzule). Uobičajene heurističke šeme koje odlučuju kada treba ponovo pokrenuti su Luby ili Rapid Restarts. Također, tehnike mašinskog učenja korištene su za optimizaciju heuristike. Ipak, potrebna je pažljivija analiza kako bi se bolje razumjelo kako komponente solvera i njihova interakcija utiču na njegovu ukupnu efikasnost.

Iako je poboljšanje solvera važan cilj, zajednica je napredovala u novim tehnikama modeliranja i metodama za certificiranje rezultata. Navest ćemo neke i pojasniti njihov značaj.

Efikasna SAT kodiranja

Kodiranja formule mogu značajno raširiti rezultujuću KNF formulu, što rezultira kvadratnim, kubnim ili još gorim dodatnim opterećenjem u poređenju sa originalnom instancom problema. Obično se suočavamo s kompromisom između prostora i jednostavnosti. Ako imamo n varijabli i dodamo m pomoćnih varijabli, potencijalni prostor pretraživanja raste od 2^n na 2^{n+m} . Ipak, u praksi dodavanje takvih varijabli može smanjiti broj klauzula, a time i ubrzati pretragu.

Pretpostavke

Predviđanje pri rješavanju je mnogo teže u praksi nego u teorijskim NP razmatranjima. S jedne strane, koliko god SAT solveri bili uspješni, oni nisu nedeterministički “vidoviti”. SAT solverima je potrebno vrijeme za rješavanje, a možda čak neće odgovoriti u razumnom roku. S druge strane, moderni SAT solveri su veoma sofisticirani. Kako bi kontrolirali dodane klauzule bez poništavanja naučenih klauzula, moderni solveri podržavaju pretpostavke, popularizirane od strane MiniSat-a, koje su dodatni literali uvedeni za predstavljanje stanja klauzula. Pokazalo se da su pretpostavke veoma raznovrsne i korisne.

Inprocessing/preprocessing Problem se pojednostavljuje prije nego što se pokrene stvarni solver. Zanimljivo je da je jedan takav zadatak predprocesiranja pokretanje ograničenog oblika Davis-Putnamove procedure na podskupu varijabli (eliminacija ograničene varijable). Međutim, moderni solveri kao što je Lingeling implementiraju sofisticiranija pravila, koja se pokreću tokom pretraživanja prije nego što se sljedeće varijable dodijele (u obradi) i uzimaju u obzir naučene klauzule.

Paralelno rješavanje

Tokom 2000-ih, Mooreov zakon je počeo da se približava svojim granicama u performansama procesora na jednoj jezgri, usporavaju poboljšanja brzine sata za čipove bazirane na siliciju. Paralelno računanje bilo je jedan od načina da se kompenzira ovaj trend, i našlo je svoj put u potrošačkom hardveru. Otprilike u isto vrijeme, paralelno računanje je također počelo da postaje modernije u SAT rješavanju, s ciljem poboljšanja ukupnih performansi obrade i rješavanja velikih instanci.

Paralelno-portfolio: Posljednjih godina, masovna paralelizacija korišćenjem grafičkih procesorskih jedinica (GPU) ili tenzorskih procesorskih jedinica (TPU) postala je popularan pristup u računarskoj nauci i ML-u. Aspekti modernog SAT rješavanja uspješno su implementirani i testirani na GPU-ovima.

Ipak, SAT rješavanje još nije imalo koristi od masovne paralelizacije.

Podijeli pa vladaj: Dijeli formulu na više formula. Radno opterećenje se dijeli između više solvera koji rade paralelno i čak mogu sinhronizirati naučene klauzule. Međutim, moderni sekvencijalni SAT solveri ne moraju iscrpljivati prostor za pretraživanje da bi pronašli rješenje ili opovrgli formulu; naučene klauzule dozvoljavaju prečice, što otežava dekomponovanje instanci zbog povećanih napora sinhronizacije za naučene klauzule. Popularna verzija je cube-and-conquer, koja particionira instancu tehnikom pogleda unaprijed na do milion podinstanci, a zatim rješava subinstance pomoću CDCL-baziranog solvera.

Ispravnost solvera i dobivanje dokaza

Rezultate ranih SAT solvera bilo je teško potvrditi. Iako je lahko provjeriti da li dodjela koju je vratio solver zadovoljava datu formulu, u prošlosti nije bila dostupna tehnika za provjeru ispravnosti ako je solver tvrdio da je instanca nezadovoljiva. Tokom prvih takmičenja moglo se samo provjeriti da li su emitovani zadaci tačni i da li solveri nisu dali nedosljedne rezultate. Ako solver A daje "zadovoljivo", a solver B daje "nezadovoljivo" na istoj formuli, provjerilo bi se da li dodjela koju daje A zaista zadovoljava formulu. Ako je tako, B je netačan; u suprotnom, A je netačan, ali se ništa ne može zaključiti o solveru B. Ispravnost solvera je adresirana intenzivnim testiranjem, uključujući fuzzing i traženje tragova koji dovode do dokaza nezadovoljivosti unutar namjenskog sistema propozicionih dokaza. Takvi tragovi se mogu automatski provjeriti pomoću alata treće strane, koji se mogu specificirati, implementirati i u potpunosti verifikirati. Odnos između CDCL pokreta i sistema propozicionih dokaza je sada dobro shvaćen. Svi relevantni sistemi dokazivanja oslanjaju se na pravilo rezolucije, koje izvodi novu klauzulu iz dvije klauzule koje sadrže tačno jedan suprotan literal. Tokom 2010-ih pojavile su se tehnike koje daju dokaze koji su kompaktni, laki za emitovanje, koji se mogu efikasno provjeriti i nude visoku izražajnost. Jedan popularan format dokaza je asimetrična tautologija brisanja i rezolucije (DRAT).

Moderna takmičenja i open source

Od 2002. godine, SAT zajednica je organizovala godišnja takmičenja koja potiču dizajn novih solvera i prikupljaju nove referentne primjere. Takmičenja nastavljaju da utiču na napredak u razvoju solvera i pružaju način za nezavisnu procenu efikasnosti prijavljenih solvera. U 2002. nekoliko solvera koristilo je CDCL arhitekturu (4 od 19). Solveri nisu bili robusni i mnogi od njih su pali tokom takmičenja - na primer, na instancama sa mnogo varijabli, mnogo klauzula ili veoma dugih klauzula. Na sreću, ovo se promijenilo sljedeće godine zbog različitih široko dostupnih mjerila i objavljenih očekivanih rezultata. U početku su mnogi solveri bili zatvorenog koda. Međutim, uspjeh besplatnog solvera MiniSat13 prebacio je zajednicu na otvoreni kod (open source). MiniSat kod je bio jednostavan za čitanje, kompaktan i dizajniran za proširivost. Implementirao je CDCL i nadmašio sve ostale solvere 2005. zbog minimiziranja naučenih klauzula i predprocesora SatELite. S vremenom je princip otvorenog koda omogućio istraživačima da implementiraju svoje ideje na vrhu najboljih dostupnih solvera i razumiju ključne detalje implementacije koji se obično ne dijele u radovima. Od 2023. godine, potomci MiniSata, kao što su Glucose,

MapleSAT i MapleCOMSPS, i dalje učestvuju u SAT takmičenju. Dok takmičenja kontinuirano podižu standarde u efikasnom rješavanju, u praksi ne postoji najbolji solver. Različiti rade različito ovisno o razmatranoj klasi mjerila ili konfiguraciji njihove interne heuristike. Ovu raznolikost mogu iskoristiti sistemi kao što je SATzilla, koji predviđaju performanse postojećih solvera i odabiru one koji najviše obećavaju. Sofisticiraniji pristup je automatski odabir i podešavanje algoritma. Alati kao što su AUTOFOLIO ili SMAC pronalaze dobre parametre za dati solver na ciljnom skupu mjerila.

Međunarodna SAT takmičenja imaju dugu tradiciju. Prvo SAT takmičenje desilo se 1992. godine, u Paderborn-u, organizovan od strane Michaela Buroa i Hansa Kleine Büninga. Drugo SAT takmičenje održano je tijekom drugog Dimacs izazova 1993. godine (Johnson i Trick 1996.). Dimacs konjunktivna normalna forma (KNF) format formulacija ulaznog formata koji se koristio na natjecanju postao je standardni ulazni format za KNF SAT solve. Još jedno SAT natjecanje održano je u Pekingu 1996. godine, koje je organizirao James Crawford. Godine 2002., Edward A. Hirsch, Daniel Le Berre i Laurent Simon pokrenuli su seriju redovito raspoređenih natjecanja: SAT natjecanje kakvo danas poznajemo.

Navesti ćemo listu takmičenja od 2002. do sada:

- 2003.g., organizatori: Daniel Le Berre i Laurent Simon.
- 2004.g., organizatori: Daniel La Berre i Laurent Simon.
- 2005.g., organizatori: Daniel La Berre i Laurent Simon.
- 2006.g., organizator: Carsten Sinz.
- 2007.g., organizatori: Daniel Le Berre, Olivier Roussel i Laurent Simon.
- 2008.g., organizator: Carsten Sinz.
- 2009.g., organizatori: Daniel Le Berre, Olivier Roussel i Laurent Simon.
- 2010.g., organizator: Carsten Sinz.
- 2011.g., organizatori: Matti Järvisalo, Daniel Le Berre i Olivier Roussel.
- 2012.g., organizatori: Adrian Balint, Anton Belov, Matti Järvisalo, Carsten Sinz.
- 2013.g., organizatori: Adrian Balint, Anton Belov, Marijn Heule, Matti Järvisalo.
- 2014.g., organizatori: Anton Belov, Daniel Diepold, Marijn Heule, Matti Järvisalo.
- 2015.g., organizatori: Tomas Balyo, Carsten Sinz, Markus Iser, Armin Biere.
- 2016.g., organizatori: Marijn Heule, Matti Järvisalo, Tomáš Balyo.

- 2017.g., organizatori: Marijn Heule, Matti Järvisalo, Tomáš Balyo.
- 2018.g., organizatori: Marijn Heule, Matti Järvisalo, Martin Suda.
- 2019.g., organizatori: Marijn Heule, Matti Järvisalo, Martin Suda.
- 2020.g., organizatori: Marijn Heule, Matti Järvisalo, Martin Suda, Markus Iser, Tomáš Balyo Nils Froleyks.
- 2021.g., organizatori: Marijn Heule, Matti Järvisalo, Martin Suda, Markus Iser, Tomáš Balyo Nils Froleyks.
- 2022.g., organizatori: Marijn Heule, Matti Järvisalo, Martin Suda, Markus Iser.
- 2023.g., organizatori: Marijn Heule, Matti Järvisalo, Martin Suda, Markus Iser.
- 2024.g., organizatori: Marijn Heule, Markus Iser, Matti Järvisalo, Martin Suda.

SAT je ključan za napredak moderne informacione tehnologije. Ipak, kritični izazovi u kompjuterskoj nauci koji se odnose na rješavanje SAT-a su još uvijek pred nama: Kako možemo dalje poboljšati paralelno pretraživanje kako bismo u potpunosti iskoristili prednosti modernog masovno paralelnog hardvera? Kako možemo poboljšati proces stvaranja dobrih kodiranja? Konačno, hoće li se SAT široko primjenjivati i na računarsku fiziku, hemiju ili nesimboličku umjetnu inteligenciju? U određenoj mjeri, revolucije SAT-a i ML-a su komplementarne. Postoji veliki potencijal u kombinovanju to dvoje.

3 Vrste SAT solvera

SAT solveri su imlementirani korištenjem različitih tehnika, u cilju poboljšanja učinkovitosti i stabilnosti. Neki koji su ključni kao komponente i tehnike u implementaciji su:

- DPLL solveri koriste povratno pretraživanja stabla odlučivanja. Osnovni način rada je na osnovu Davis–Putnam–Logemann–Loveland algoritma. Mnogi moderni pristupi praktičnom rješavanju SAT-a su izvedeni iz DPPL algoritma.
- CDCL (Conflict-Driven Clause Learning) je tehnika koja je nadopunila DPPL-algoritam. Umjesto donošenja odluke o vrijednostima varijabli CDCL solver "uči" iz konflikta tokom pretrage i dodaju nučene klauze koje pomažu u izbjegavanju sličnih konflikta u budućnosti.

Paralelni SAT-solving dolaze u tri kategorije algoritama: portfolio, zavadi i vladaj i lokalna pretraga.

- Portfolio je skup različitih algoritama ili konfiguracija istog algoritma. Svi solveri u paralelnom portfoliou rade na različitim procesorima kako bi riješili isti problem. Ako se jedan solver završi, solver portfolioa prijavljuje da je problem zadovoljavajući ili nezadovoljiv na osnovu ovog jednog solvera.
- Za razliku od paralelnih portfolioa, zavadi pa vladaj nastoje da podijele prostor pretraživanja između elemenata obrade. Zavadi i vladaj već primjenjuju tehniku podjele prostora za pretragu pa je njihovo proširenje ka paralelnom algoritmu jednostavno. Međutim zbog tehnika kao što je propagacija jedinica, nakon podjele, parcijalni problemi mogu se značajno razlikovati u složenosti.
- Jedina strategija prema algoritmu paralelnog lokalnog pretraživanja za rješavanje SAT-a je pokušaj višestrukih okretaja varijabli istovremeno na različitim procesorskim jedinicama. Drugi je primjena gore spomenutog portfolio pristupa, međutim dijeljenje klauzule nije moguće jer lokalni solveri pretraživanja ne proizvode klauzule.

Slučajni pristupi (Randomized approaches) ovo su algoritmi koji nisu dio DPLL porodice, uključuju stohastičke algoritme lokalnog pretraživanja. Jedan od primjera je WalkSAT. Stohastičke metode nastoje da pronađu zadovoljavajuće interpretacije ali ne mogu dedukovati da SAT instanca nije zadovoljavajuća, za razliku od suprotnih potpunih algoritama, kao DPPL.

3.1 DPPL-algoritam

Davis–Putnam–Logemann–Loveland (DPLL) algoritam je kompletni algoritam pretraživanja zasnovan na povratnom tragu za odlučivanje o zadovoljivosti proposicionih logičkih formula u konjunktivnom normalnom obliku, odnosno za rješavanje KNF-SAT problema.

Ovaj algoritam je uveden 1961. od strane Martin Davis , George Logemann i Donald W. Loveland i on označava poboljšanje ranjeg Davis-Putnam algoritma, koji je zasnovan na rezoluciji koji su razvili Davis i Hilary Putnam.

DDPL je jednostavni algoritam vraćanja nazad (backtracking) koji radi na odabiru literala, zatim dodjele istinitosti tim literalima. Pojednostavljuje formulu i onda rekurzivno provjerava da li je pojednostavljena formula zadovoljavajuća. Ako je ovo slučaj, originalna formula je zadovoljavajuća, u suprotnom rekurzivna promjena je izvršena sa pretpostavkom suprotnih istinitih vrijednosti. Ovo je poznato kao pravilo cijepanja, jer cijepa problem na dva jednostavnija sub-problema. Proces simplifikacije esencijalno uklanja sve klauzule koje postaju tačne pri dodjeli iz formule i sve literali koji postaju netačni iz preostalih klauzula.

DPPL algoritam je poboljššan za razliku od algoritma za vraćanje u nazad korištenjem slijedećih pravila pri svakom koraku:

- Jedinično širenje: Ako je klauzula jedinična klauzula , tj. sadrži samo jedan nedodijeljeni literal, ova klauzula može biti zadovoljena samo dodjeljivanjem potrebne vrijednosti da bi ovaj literal bio istinit.
- Čista bukvalna eliminacija: Ako se proposiciona varijabla pojavljuje sa samo jednim polaritetom u formuli, naziva se čista . Čisti literal se uvijek može dodijeliti na način koji čini da su sve klauzule koje ga sadrže istinite. Dakle, kada se dodijeli na takav način, ove klauzule više ne ograničavaju pretragu i mogu se izbrisati.

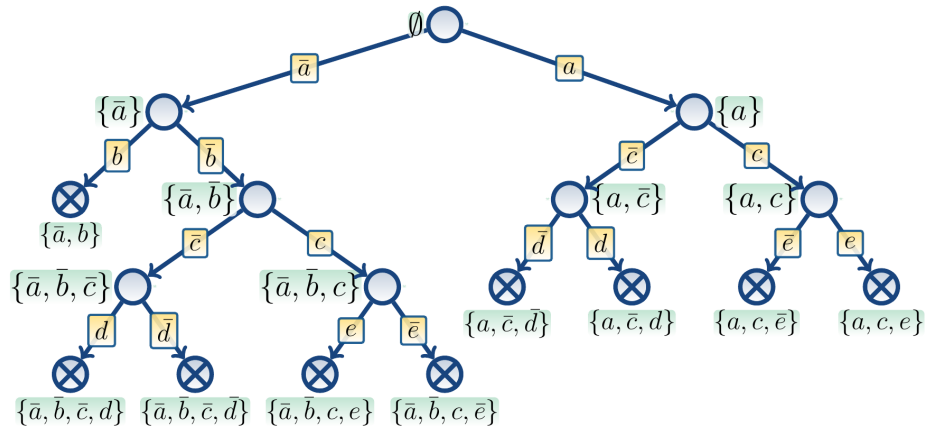


Figure 1: DPPL

3.2 CDCL-algoritam

Učenje klauzula vođeno konfliktom (CDCL) je SAT algoritam ,inspiriran je DPPL solverima. Učenje klauzule vođeno konfliktom predložili su Marques-Silva i Karem A. Sakallah (1996, 1999) i Bayardo i Schrag (1997). Glavna razlika između CDCL-a i DPPL-a je što kod CDCL-a pretraživanje unazad nije po hronološkom redoslijedu. Također CDCL SAT solver uključuje i dodatni broj tehnika:

- Učenje novih klauzula iz sukoba tokom pretraživanja unazad
- Iskorištavanje strukture sukoba tokom učenja klauzule
- Upotreba lijenih struktura podataka za predstavljanje formula
- Heuristike grananja moraju imati niske računske troškove i moraju primati povratnu informaciju od pretraživanja unazad
- Periodično ponovno pokretanje pretraživanja nazad

Učenje klauzule vođeno konfliktom (CDCL) solveri su široko iskorišteni u mnogim segmentima i efikasni su praksi. Od svog nastanka sredinom 90-ih, CDCL SAT solveri primijenjeni su, u mnogim slučajevima sa izuzetnim uspjehom, na brojne praktične primjene. Primjeri aplikacija uključuju hardverski i softverski model provjeravanje, planiranje, provjera ekvivalencije, bioinformatika, hardver i soft-ver generisanje test obrazaca, zavisnosti od softverskih paketa i kriptografija.

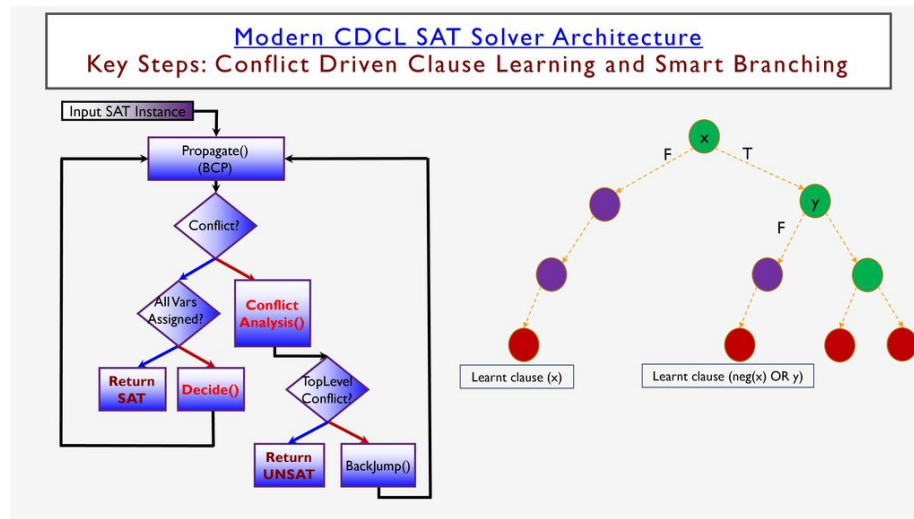


Figure 2: CDCL

3.3 Paralelni SAT-solveri

Zasnovane na jednojezgrenim arhitekturama, prvi paralelni SAT rješavači bili su zasnovani na mrežnoj komunikaciji. Sa pojavom višejezgrenih CPU-a, za komunikaciju je korištena i zajednička memorija. Sa prelaskom sa DPLL na CDCL solve, uključujući ponovno pokretanje, razvijene su različite tehnike za paralelizaciju algoritama za rješavanje SAT-instanci. 2006. godine, prvi pregled paralelnih SAT-rešavača je predstavljen.

Rekurzivna primjena split pravila u DPLL algoritmu pruža prirodan način za paralelizaciju pretrage. U početku, svaki računarski čvor prima datu formulu. Nakon toga se saopštavaju samo djelomične interpretacije tako da svaki računski čvor prima drugačiju djelimičnu interpretaciju. Prvi paralelni SAT rješavači koristili su jednojezgrene procesore koji su komunicirali putem mreže. Kada su dijeljene memorijske arhitekture postale dostupne, korištena je komunikacija dijeljene memorije. Uspjeh prvog SAT rješavanja zasnovanog na CDCL-u doveo je do dramatične promjene. Zbog dodavanja naučenih klauzula, prostor pretraživanja se prelazi mnogo manje nego u DPLL-u.

3.4 Portfolio

Zbog velikog broja ponovnog pokretanja u sekvencijalnim SAT solverima dovelo je do prelaska sa kooperativnog paralelizma. Umjesto toga, prešli su na strategiju cijepanja prostora za pretraživanje na kompetitivni paralelizam gdje svi paralelni solveri istovremeno nastoje pronaći rješenje za istu SAT instancu, nazvanu portfolio solverima. Jedan od popularnih takvih rješavača je MANYSAT, koji koristi različite tehnike poput heuristika ponovnog pokretanja, odlučivanja i učenja na četiri paralelne instance.

3.5 Zavadi pa vladaj

Ovaj tip u paralelnom SAT solveru se temelji na razdvajanju prostora za pretragu u podrpostore koji su predani za izvršenje drugim "radnicima". Ako je podprostor dokazan SAT onda je inicijalna formula također SAT. U uprotnom formula je UNSAT ako su svi podprostori isti UNSAT. Izazovni dijelovi zavadi pa vladaj metode su: podjela prostora koji se pretražuje, ravnomjerno raspoređivanje poslova i razmjena već naučenih klauzula.

3.6 Lokalno pretraživanje

Algoritmi lokalnog pretraživanja su svrstani u jedne od standardnih metoda za rješavanje kombinatorike i problema iz različitih oblasti umjetne inteligencije, kao i operativnih istraživanja. Neki od najznačajnijih i najuspješnijih algoritama su zasnovani na stohastičkom lokalnom pretraživanju. Veliki broj sličnih algoritama su u posljednjih 10 godina su predloženi i istraženi. Ovaj algoritam je 1992. uveden od Selman-a, Levesque-a i Mitchell-a koji su bazirani na stohastičkom lokalnom "hill-climbing" koji je prikazan da ima veću efikasnost od

najsavremenijih sistematskih SAT algoritama, kao ASAT.OD tada su predloženi brojni drugi tipovo stohastičkog lokalnog pretraživanja (SLS) za SAT. Danas najsavremeniji SLS (Stochastic local search) algoritmi mogu riješiti probleme i sa nekoliko hiljada varijabli kao i SAT kodirane probleme iz drugih oblasti.

3.7 Slučajni pristup (Randomized approaches)

Slučajni pristupi SAT solverima su tehnike koje koriste nasumične strategije za pretragu rješenja u problemima zadovoljivosti booleovih formula (SAT problema). Ove tehnike često koriste nasumično generirane vrijednosti kao početne tačke pretrage, te kombinuju nasumične izbore s heuristikama kako bi povećali šanse za pronalazak rješenja. Ovi pristupi obično se koriste za rješavanje NP-teških problema poput SAT-a, gdje nema efikasnog algoritma koji može pronaći rješenje u polinomijalnom vremenu. Slučajni pristupi su često korisni kada je prostor stanja velik i kada se traži brzo rješenje, ali nisu garantirani da će pronaći optimalno rješenje.

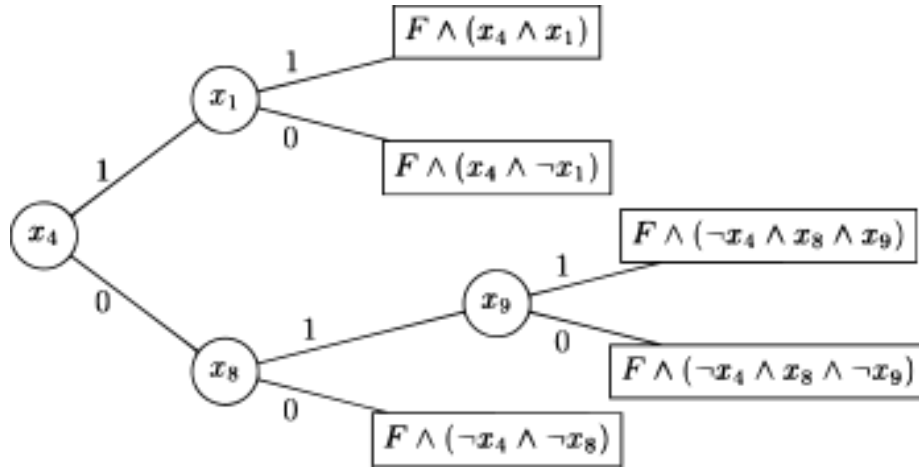


Figure 3:

4 Boolean operatori

Logički Booleovi operatori su standardne funkcije koje uzimaju jednu ili više binarnih varijabli kao ulaz i vraćaju jedan binarni izlaz. Stoga se mogu definisati tabelama istinitosti u kojima nabrajamo svaku kombinaciju ulaza i definiramo izlaz za svaku (slika 1). Uobičajeni logički operatori uključuju:

- **Operator OR/ILI** se zapisuje kao $\vee$ i uzima dva literala A i B, te vraća istinu ako su jedan ili oba literala istinita, a u suprotnom vraća neistinu.
- **Operator AND/I** je napisan kao $\wedge$ i uzima dva literala A i B. Vraća istinu ako su oba ulaza tačna i netačna istovremeno, dok u suprotnom vraća neistinu.
- **Operator IMPLIKACIJE** se piše kao $\Rightarrow$ i procjenjuje da li su dva ulaza u skladu s naredbom 'ako A, onda B'. Izjava se ne poštuje samo kada je A isinito, a B neistinito, u svim ostalim slučajevima ovaj operator vraća istinu.
- **Operator EKVIVALENCIJE** se piše kao $\Leftrightarrow$ i uzima dva ulaza A i B. Vraća se istina ako su oba ulaza ili tačna ili netačna, u suprotnom ovaj operator daje neistinu.
- **Operator NEGACIJE** se piše kao $\neg$ i to je unarni operator, dakle uzima jedan ulaz. Vraća se istina ako je ulaz neistinit i obrnuto.

OR			AND			IMPLICATION			EQUIVALENCE			NOT	
x_1	x_2	$x_1 \vee x_2$	x_1	x_2	$x_1 \wedge x_2$	x_1	x_2	$x_1 \Rightarrow x_2$	x_1	x_2	$x_1 \Leftrightarrow x_2$	x_1	$\neg x_1$
0	0	0	0	0	0	0	0	1	0	0	1	0	1
0	1	1	0	1	0	0	1	1	0	1	0	1	0
1	0	1	1	0	0	1	0	0	1	0	0		
1	1	1	1	1	1	1	1	1	1	1	1		

Figure 4: Tablica istine za operatore ILI, I, IMPLIKACIJA, EKVIVALENCIJA i NEGACIJA

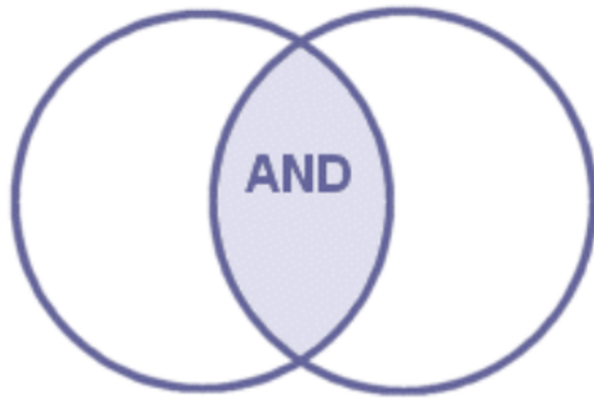


Figure 5: Operator I predstavljen Veneovim dijagramom



Figure 6: Operator ILI predstavljen Veneovi dijagramom

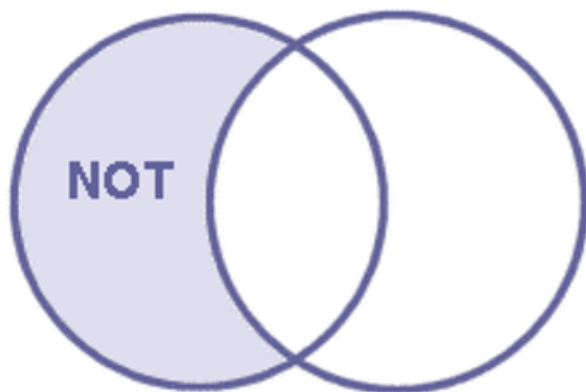


Figure 7: Operator NEGACIJE predstavljen Veneovim dijagramom

5 Zadovoljivost Booleove logike i SAT solveri

Problem Booleove zadovoljivosti postavlja pitanje da li postoji bar jedna kombinacija x_i , koja će kao rezultat dati tačnu tvrdnju. SAT solver uzima logičku Booleovu formulu kao ulaz i vraća *SAT* (*satisfiability* / *zadovoljenu tvrdnju*) ukoliko pronađe kombinaciju varijabli za koje tvrdnja može biti zadovoljena, to jest *UNSAT* (*unsatisfiability* / *ne zadovoljava tvrdnju*), ukoliko može pokazati da takva kombinacija ne postoji. Osim toga, ponekad se može vratiti izlaz bez odgovora ako ne može utvrditi da li je problem SAT ili UNSAT.

Bilo koja formula se može ponovo napisati kao konjunkcija disjunkcija, a ovaj oblik je poznat pod nazivom **konjunktivni normalni oblik (KNF)**. Da bismo riješili SAT problem, prvo pretvaramo Booleovu logičku formulu u KNF, jer je to standardni oblik koji je više podložan algoritamskoj manipulaciji, lahko je otkriti konflikt ukoliko postoji i lahko je zapamtiti dijelove zadatka koji ne funkcionišu, kada je sam zadatak napisan u takvoj formi. Najosnovniji pristup je kreiranje ekvivalentne formule koristeći De-Morganove zakone, distributivni zakon i činjenicu da se dvije negacije poništavaju. Ovaj pristup ima dvije prednosti: jednu, jednostavan je i očigledno ispravan. Drugo, ne uvodi nove varijable. Međutim, ima jedan značajan nedostatak: neke formule dovode do eksponencijalno velike KNF konverzije. Drugi pristup je kreiranje ekvizadovoljive KNF formule, što znači da su dvije formule, f_1 i f_2 , jednakoizadovoljive kada je f_1 zadovoljen znači da je i f_2 zadovoljen i obrnuto.

$\neg\neg\alpha$	α
$\alpha \implies \beta$	$\neg\alpha \vee \beta$
$\neg(\alpha \wedge \beta)$	$\neg\alpha \vee \neg\beta$
$\neg(\neg\alpha \wedge \neg\beta)$	$\alpha \vee \beta$
$(\alpha \wedge \beta) \vee \gamma$	$(\alpha \vee \gamma) \wedge (\beta \vee \gamma)$
$\alpha \iff \beta$	$(\alpha \implies \beta) \wedge (\alpha \impliedby \beta)$

Figure 8: Tablica ekvivalencije (originalna klauzula lijevo, ekvivalentna klauzula desno)

Grativni elementi normalnih formi logičkih iskaza:

- **Literal** predstavlja ili varijablu (atom) p propozicijske logike ili njenu negaciju p. Literal se često naziva osnovnim iskazom ili rečenicom.
- **Disjunktivna klauzula (elementarna disjunkcija)** predstavlja izraz dobijen iz konačnog skupa literala i koji je tačan ako je barem jedan od literala tačan. Primjer: $A \vee B \vee D$
- **Konjunktivna klauzula (elementarna konjunkcija)** predstavlja izraz dobijen iz konačnog skupa literala i koji je tačan ako i samo ako je svaki od literala tačan. Primjer: ABD^1

Razlika između varijable i literala je u tome što je literal konkretna "evaluacija" varijable unutar klauzule.

Kao naprimjer, formula $(A \vee B \vee \neg C) \wedge (\neg A \vee \neg B)$ sadrži tri varijable, A, B i C, ali također sadrži pet literala A, $\neg A$, B, $\neg B$ i $\neg C$.

Definicija 1: Disjunktivna forma je bilo koji izraz koji predstavlja disjunkciju više prostih izraza. Pr. $B \vee AC \vee BC$
Definicija 2: Disjunktivna normalna forma predstavlja normalizaciju disjunktivne forme u disjunkciju elementarnih konjunkcija. (DNF ili SOP) Pr. $A \vee BC \vee BC$ Analogno se definišu konjunktivna i konjunktivna normalna forma.

¹Možemo povući paralelu između Booleovih operatora i osnovnih aritmetičkih operatora koje koristimo u svakodnevno za računanje: slično operatoru sabiranja, u Booleovoj algebri se koristi operator disjunkcije, to jest logičko ILI ($\vee$); slično operatoru množenja, u Booleovoj algebri se koristi operator konjunkcije, to jest logičko I ($\wedge$), te zato, kao i kod klasičnog množenja, radi lakšeg pisanja, umjesto $A*B*C$, možemo pisati samo ABC, analogno je i sa Booleovim operatorima.

6 Tseitinova transformacija

Tseitinova transformacija pretvara proizvoljnu logičku formulu u konjunktivni normalni oblik. Pristup transformaciji se vrši u par koraka:

1. povezivanje novih varijabli sa poddijelovima formule koristeći logičke relacije ekvivalencije;
2. formula se ponovo formuliše logički spajanje ovih novih varijabli zajedno konjunkcijom, i konačno
3. manipulisanje svakom od relacija ekvivalencije tako da one same budu u konjunktivnom normalnom obliku.

Ovaj proces je najlakše razumjeti na konkretnom primjeru. Razmotrimo konverziju formule:

$$\phi = ((x_1 \vee x_2) \leftrightarrow x_3) \rightarrow (\neg x_4)$$

Korak 1: Povezujemo nove binarne varijable sa poddijelovima originalne formule koristeći operator ekvivalencije:

$$y_1 \Leftrightarrow (x_1 \vee x_2) \quad y_2 \Leftrightarrow (y_1 \Leftrightarrow x_3) \quad y_3 \Leftrightarrow \neg x_4 \quad y_4 \Leftrightarrow (y_2 \Rightarrow y_3)$$

Korak 2: Ponovo navodimo formulu u smislu ovih odnosa. Potpuna origi-

nalna izjava je sada predstavljena sa y_4 zajedno sa definicijama y_1, y_2, y_3, y_4 u jednačini 4. Dakle, izjava je kada kombinujemo sve ove relacije sa logičkim I relacijama. Radeći unazad dobijamo:

$$\phi = y_4 \wedge (y_4 \Leftrightarrow (y_2 \Rightarrow y_3)) \wedge (y_3 \Leftrightarrow \neg x_4) \wedge (y_2 \Leftrightarrow (y_1 \Leftrightarrow x_3)) \wedge (y_1 \Leftrightarrow (x_1 \vee x_2))$$

Korak 3: Pretvorimo svaki od ovih pojedinačnih pojmova u konjunktivni normalni oblik. U praksi postoji način za svaki tip operatora:

$$a \Leftrightarrow (\neg b) = (a \vee b) \wedge (\neg a \vee \neg b)$$

$$a \Leftrightarrow (b \vee c) = (a \vee \neg b) \wedge (a \vee \neg c) \wedge (\neg a \vee b \vee c)$$

$$a \Leftrightarrow (b \wedge c) = (\neg a \vee b) \wedge (\neg a \vee c) \wedge (a \vee \neg b \vee \neg c)$$

$$a \Leftrightarrow (b \Rightarrow c) = (a \vee b) \wedge (a \vee \neg c) \wedge (\neg a \vee \neg b \vee c)$$

$$a \Leftrightarrow (b \Leftrightarrow c) = (\neg a \vee \neg b \vee c) \wedge (\neg a \vee b \vee \neg c) \wedge (a \vee \neg b \vee \neg c) \wedge (a \vee b \vee c)$$

7 SAT konstrukcije

SAT konstrukcije se mogu smatrati potprogramima za Booleove logičke izraze. Uobičajena situacija je da imamo skup varijabli $x_1, x_2, x_3, \dots$ i želimo da nametnemo kolektivno ograničenje na njihove vrijednosti. U ovom dijelu ćemo razgovarati o tome kako da primijenimo ograničenja da su sva ista, da je tačno jedno od njih tačno, da nije više od K od njih istinito ili da je tačno K od njih.

7.0.1 Jednakost (Same)

Da bismo nametnuli ograničenje da je skup varijabli x_1, x_2, x_3 ili sve istinito ili sve netačno, jednostavno uzimamo logičko ILI ova dva slučaja tako da imamo:
 $[x_1, x_2, x_3](x_1 \wedge x_2 \wedge x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge \neg x_3)$

Treba uzeti u obzir da ovo nije u konjunktivnom normalnom obliku (AND i OR su pogrešni načini), ali bi se moglo pretvoriti u KNF putem Tseitinove transformacije.

7.0.2 Tačno jedan (Exactly one)

Da bismo nametnuli ograničenje da je samo jedna iz skupa varijabli x_1, x_2, x_3 istinita, a druge dvije netačne, dodajemo dva ograničenja. Prvo osiguravamo da je barem jedna varijabla istinita logičkim ILI spajanjem varijabli:

$$\phi_1 x_1 \vee x_2 \vee x_3$$

Zatim dodajemo ograničenje koje pokazuje da oba člana bilo kojeg para varijabli ne mogu biti istovremeno istinita:

$$[x_1, x_2, x_3] = \phi_1 \wedge \neg(x_1 \wedge x_2) \wedge \neg(x_1 \wedge x_3) \wedge \neg(x_2 \wedge x_3)$$

7.0.3 Bar K, Manje od K, Tačno K (At least K, Less than K, Exactly K)

Postoji mnogo standardnih načina da se nametne ograničenje da je najmanje K skupa varijabli istinito. Predstavimo jednu metodu koja je pojednostavljena verzija kodiranja *sekvencijalnog brojača*. Ideja je jasna. Ako imamo J varijabli $x_1, x_2, \dots, x_J$ i želimo da testiramo da li je K ili više tačnih, konstruišemo JxK matricu koja sadrži nove binarne varijable $r_{J,K}$ (tabele b) i d)). j-ti red tabele sadrži broj pravih elemenata koje smo videli u $x_1, \dots, x_j$. Dakle, ako smo vidjeli 3 varijable koje su istinite u prvim j elemenata, j-ti red će početi sa 3 tačna elementa i završiti sa K-3 elementa.

a)

$$x = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

b)

$$R = \left[\begin{array}{c|cccccc} K & & & & & \\ \hline 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 \end{array} \right]$$

c)

$$x = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

d)

$$R = \left[\begin{array}{c|cccccc} K & & & & & & \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \end{array} \right]$$

Ako postoji najmanje K varijabli, onda donja desna varijabla $r_{J,K}$ u ovoj tabeli mora biti tačna i tako bismo u praksi dodali klauzulu $(r_{J,K})$ s lausom navodeći da ovaj donji desni element mora biti istinit da bi se nametnulo ograničenje. Kada je ovaj element netačan, solver će tražiti drugo rješenje gdje x ima najmanje K elemenata ili će vratiti UNSAT ako ga ne može pronaći. Po istoj logici, da bismo nametnuli ograničenje da ima manje od K elemenata, dodajemo klauzulu $r_{J,K}$ u kojoj se navodi da je donja desna varijabla neistinita.

Tabela pod d) nam također pokazuje kako da ograničimo podatke da imaju tačno K pravih vrijednosti. Ovdje bismo trebali konstruisati tabelu sa $K+1$ redova. Očekujemo da je donji desni element lažan (nema $K+1$ tačnih vrijednosti), ali da je element lijevo od ovoga istinit (postoji najmanje K tačnih vrijednosti). Dakle, možemo dodati klauzulu $(r_{J,K+1} \wedge r_{J,K})$. Slika 6 pruža više detalja o tome kako dodajemo dodatne klauzule u SAT formulu koje grade ove tabele.

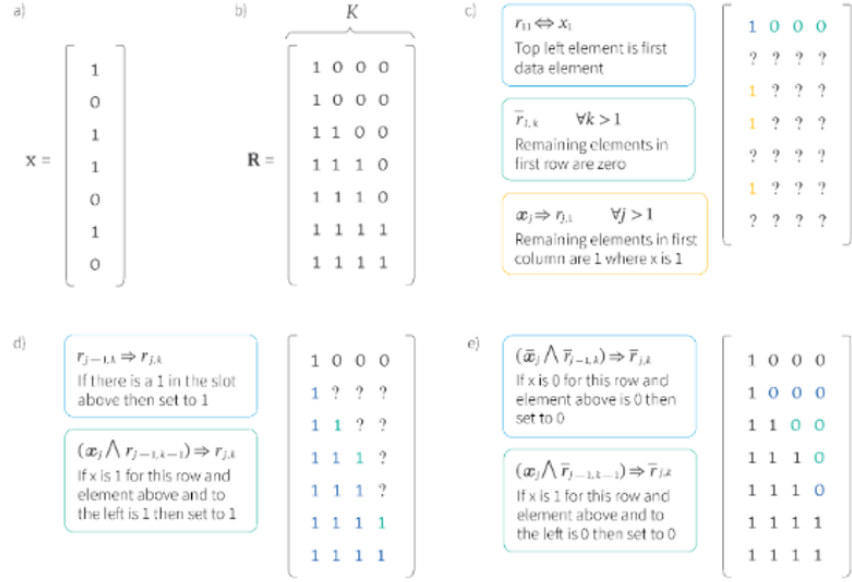


Figure 9: SAT konstrukcija za 'najmanje K' ograničenje. a) Razmotrimo vektor podataka x dužine 7 sa elementima x_j . b) Da bismo testirali da li ima najmanje $K=4$ elementa koji su tačni, konstruišemo matricu R 7×4 gdje j -ti red broji koliko smo istinitih vrijednosti vidjeli do x uključujući j -ti element od x . c-e) Inkrementalna konstrukcija tabele. Kada započnemo, svi elementi u tabeli su nesigurni (predstavljani sa '?'). Povežemo konjuncijom zajedno niz klauzula koje ih zajedno ograničavaju da formiraju tabelu. U svakom slučaju boja ograničenja odgovara boji elemenata koje ograničava.

Sa ovim SAT konstrukcijama, sada ćemo predstaviti dva komplementarna načina razmišljanja o SAT aplikacijama. Razmotrit ćemo SAT u smislu problema sa zadovoljenjem ograničenja, a u odjeljku koji slijedi raspravljat ćemo o njemu u smislu prilagođavanja modela.

8 Bojenje grafika

U problemu bojenja grafika (slika 9) dat nam je grafik koji se sastoji od skupa vrhova i linija. Želimo da povežemo svaki vrh s bojom na takav način da svaki par vrhova povezanih linijom ima različite boje. Možda bismo također željeli znati koliko je boja potrebno da bismo pronašli valjano rješenje. Imajmo na umu da ovo odgovara našem opisu generičkog problema zadovoljenja ograničenja;

postoji veliki broj mogućih dodjela boja, ali mnoge od njih su isključene zbog ograničenja da susjedne boje moraju biti različite.

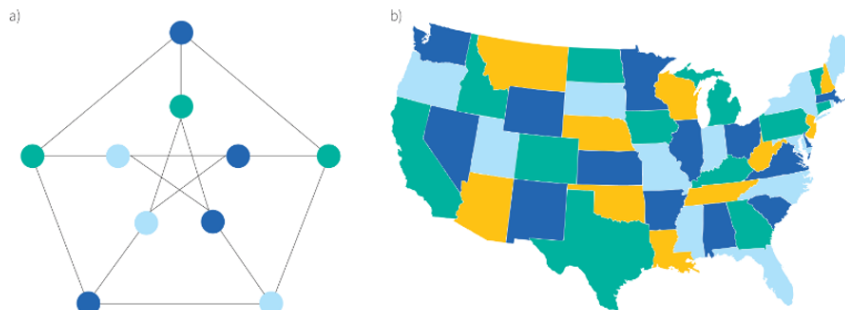


Figure 10: Bojenje grafikona. a) Dat nam je grafik koji se sastoji od skupa vrhova i linija koje ih povezuju. Želimo obojiti vrhove na način da dva vrha imaju različite boje ako su povezani ivicom. Problem bojenja grafika ima za cilj da uspostavi najmanji broj boja za koji je to moguće i vrati zadovoljavajuću dodjelu boja. Za ovaj grafik se to može postići sa tri boje. b) Problem bojenja grafika ima praktičnu primjenu u kartama bojanja. Ovdje svaka američka država odgovara vrhu i dodajemo liniju ako su dva susjedna stanja. Dokazano je da sve takve 2D karte zahtijevaju najviše četiri boje.

Da bismo ovo kodirali kao SAT problem, izabrat ćemo broj boja C za testiranje. Zatim kreiramo binarne varijable $x_{c,v}$ koje će biti istinite ako je vrh v obojen bojom c . Zatim kodiramo ograničenje da svaki vrh može imati samo jednu boju koristeći konstrukciju $\text{ExactlyOne}(x)$ iz jednačine 12. Također dodajemo ograničenja kako bismo osigurali da susjedi imaju različite boje. Formalno to znači da $x_{c,v} \rightarrow x_{c',v'}$ za svaku boju c i komšiju v' vrha v . Nakon što smo postavili problem, pokrećemo SAT solver. Ako vrati UNSAT, to znači da nam treba više boja. Ako vrati SAT sa konkretnim bojenjem, onda imamo odgovor. Možemo pronaći minimalni broj potrebnih boja korištenjem binarnog pretraživanja preko broja boja da pronađemo tačku gdje se problem mijenja iz SAT u UNSAT.

Problem bojenja grafika je prilično vještački primjer informatike, ali mnogi problemi iz stvarnog svijeta mogu se na sličan način izraziti u terminima zadovoljivosti. Na primjer, zakazivanje kurseva na univerzitetu. Postoji veliki broj profesora, od kojih svaki predaje nekoliko različitih predmeta. Na raspolaganju je nekoliko učionica i nekoliko mogućih termina u svakoj učionici. Konačno, tu su sami studenti, od kojih je svaki upisan na različite podskupove kurseva. Možemo koristiti SAT mašineriju da odlučimo koji predmet će se predavati u kojoj učionici i u kojem terminu, tako da ne dođe do međusobnog preklapanja termina u istim učionicama.

U praksi se to radi definisanjem binarnih varijabli koje opisuju poznate relacije između stvarnih veličina. Naprimjer, možemo imati varijable $x_{i,j}$ koje pokazuju da student i pohađa kurs j . Zatim kodiramo relevantna ograničenja: nijedan nastavnik ne može istovremeno predavati dva razreda, nijedan učenik ne može istovremeno biti u dva razreda, nijedna prostorija ne može ugostiti više od jednog razreda istovremeno, itd.

9 Primjena

Moderni SAT solveri su u stanju jako efikasno riješiti brojne probleme iz raznih oblasti naše svakodnevnice. Njihovu implementaciju značajno olakšava postojanje velikog broja alata za rukovanje SAT solverima čije je korištenje omogućeno u raznim programskim jezicima, a jedan od takvih alata je Z3.

Z3 je moćan alat za rješavanje problema iz raznih oblasti, a između ostalog, pogodan je i za rješavanje SAT problema. Možemo ga koristiti u nekom online okruženju, a možemo i instalirati u svom lokalnom okruženju. Ako želimo koristiti z3 kao biblioteku u programskom jeziku Python, ovu biblioteku instaliramo jednostavnom naredbom „pip install z3-solver“ u terminalu.

Z3 biblioteka u Pythonu nudi nam konstruktore za konjunkciju, disjunkciju, negaciju, implikaciju i ekskluzivnu disjunkciju, dok za ekvivalenciju koristimo operator „==“. Ove funkcionalnosti primjenjujemo na Bool varijablama iz biblioteke z3 kako bismo kreirali klauze. Te klauze dodajemo u solver pomoću funkcije `add()` koja se primjenjuje nad solverom, a koja kao parametar prima klauzu koju dodajemo. Od klauza dodanih u solver kreira se konjunktivna normalna forma, a poziv funkcije `check()` nad kreiranim solverom vraća kao rezultat vrijednost „sat“ ili „unsat“ u zavisnosti od proslijeđenih klauza. Ako funkcija `check()` vraća vrijednost „sat“, nad solverom možemo primijeniti i funkciju `model()` koja na izlazu vraća odgovarajuće vrijednosti ulaznih varijabli za koje je izraz proslijeđen solveru zadovoljavajući. Pri korištenju ove funkcije moramo biti pažljivi iz razloga što poziv ove funkcije nad solverom koji vraća vrijednost „unsat“ uzrokuje grešku zbog koje program krahira. Važno je još napomenuti da `z3.Bool` varijable nisu isto što i obične bool varijable. Običnim bool varijablama dodjeljujemo vrijednosti `True` ili `False` i kao takve ih koristimo, dok `z3.Bool` varijable zapravo predstavljaju literale koje koristimo bez dodjele vrijednosti.

Sad kad smo se upoznali sa osnovnim funkcionalnostima z3 biblioteke u Pythonu, možemo preći na neke konkretne primjere SAT solvera.

9.1 Primjer 1

Pretpostavimo da ako je predstava dobra, onda nije loš tekst, ili su dobri režiser i glumci. Također pretpostavimo da je predstava dobra i da su glumci dobri. Pokazati da iz toga slijedi da tekst nije loš ili da je režiser dobar.

Odgovarajućom interpretacijom ovih izraza kroz jezik formalne logike, ovaj dokaz svodi se na dokazivanje toga da je logički izraz $(A \Rightarrow ((\neg B) \vee CD)) \wedge AD \Rightarrow ((\neg B) \vee C)$ tautologija. Primijetimo da nije dovoljno samo pokazati da je ovaj izraz zadovoljavajući, nego mora biti zadovoljavajući za sve svoje modele, odnosno za sve moguće kombinacije vrijednosti ulaznih varijabli izlaz mora biti "sat".

```
1  # uvođenje biblioteke z3
2  import z3
3  # kreiranje literala
4  a = z3.Bool('a')
5  b = z3.Bool('b')
6  c = z3.Bool('c')
7  d = z3.Bool('d')
8  # kreiranje klauza
9  k1 = z3.Implies(a, z3.Or(z3.Not(b), z3.And(c, d)))
10 k2 = z3.And(a, d)
11 k3 = z3.Or(z3.Not(b), c)
12 # kreiranje traženog logičkog izraza
13 k = z3.Implies(z3.And(k1, k2), k3)
14 # kreiranje solvera i dodavanje kreiranog izraza u solver
15 s = z3.Solver()
16 s.add(k)
17 # provjera zadovoljivosti datog izraza
18 provjera = s.check()
19 print("PROVJERA: ", provjera)
20
```

Figure 11: Primjer 1 - realizacija, prvi dio

```

20
21 # kreiranje liste prvih 2^4 = 16 binarnih brojeva zapisanih na 4 bita
22 # pomoću ove liste ćemo proći kroz sve moguće kombinacije ulaznih vrijednosti literala
23 binarni = ['0000', '0001', '0010', '0011', '0100', '0101', '0110', '0111',
24           '1000', '1001', '1010', '1011', '1100', '1101', '1110', '1111']
25
26 print("TABLICA ISTINE:")
27 print()
28 print("-----")
29 print("| a | b | c | d | IZRAZ |")
30 print("-----")
31
32 for i in range(16):
33     # kreiranje klasičnih bool varijabli
34     # na osnovu liste binarnih brojeva varijable poprimaju vrijednosti "True" ili "False"
35     A = bool(int(binarni[i][0]))
36     B = bool(int(binarni[i][1]))
37     C = bool(int(binarni[i][2]))
38     D = bool(int(binarni[i][3]))
39     # kreiranje klauza za konkretne vrijednosti ulaznih varijabli
40     k1 = z3.Implies(A, z3.Or(z3.Not(B), z3.And(C, D)))
41     k2 = z3.And(A, D)
42     k3 = z3.Or(z3.Not(B), C)
43     # kreiranje logičkog izraza za konkretne vrijednosti ulaznih varijabli
44     k = z3.Implies(z3.And(k1, k2), k3)
45     # kreiranje solvera i dodavanje kreiranog izraza u solver
46     s1 = z3.Solver()
47     s1.add(k)
48     # ispisivanje vrijednosti varijabli i samog izraza u tablicu istine
49     print("|", "{:<5}".format(A), "|", "{:<5}".format(B), "|", "{:<5}".format(C),
50           "|", "{:<5}".format(D), "|", end = " ")
51     if s1.check() == z3.sat:
52         print(" 1 |")
53     else:
54         print(" 0 |")
55     print("-----")
56

```

Figure 12: Primjer 1 - realizacija, drugi dio

Implementacija logičkog izraza $(A \Rightarrow ((\neg B) \vee CD))AD \Rightarrow ((\neg B) \vee C)$ data je na prethodne dvije slike, dok je na slici ispod data tabela istine za ovaj primjer. Vidimo da izraz zaista za sve vrijednosti ulaznih varijabli dostiže vrijednost "True", što nam je i bio cilj pokazati. Dakle, ovaj logički izraz je tautologija, a samim tim i "sat", ali da bismo bolje razumjeli razliku između ova dva pojma, poslužiti će nam sljedeći primjer.

PROVJERA: sat					
TABLICA ISTINE:					
	a	b	c	d	IZRAZ
	0	0	0	0	1
	0	0	0	1	1
	0	0	1	0	1
	0	0	1	1	1
	0	1	0	0	1
	0	1	0	1	1
	0	1	1	0	1
	0	1	1	1	1
	1	0	0	0	1
	1	0	0	1	1
	1	0	1	0	1
	1	0	1	1	1
	1	1	0	0	1
	1	1	0	1	1
	1	1	1	0	1
	1	1	1	1	1

Figure 13: Primjer 1 - ispis

9.2 Primjer 2

Student pred ispitni rok ima sljedeće razmišljanje:

„Ako položim predmet A ili ako padnem predmet B, onda ću ili položiti predmet C ili pasti predmet D. Položit ću predmet C ako i samo ako položim predmete A i B. Ako padnem predmet B, onda ću ili pasti predmet C ili pasti predmet A. Siguran sam da ću pasti predmet D, osim ako padnem predmet B i položim predmet C.“

Ispitati konzistentnost ovog rezonovanja i ako je ono konzistentno pronaći sve moguće scenarije koji se uklapaju u ovo rezonovanje.

Kao i u prethodnom primjeru, potrebno je ovu izjavu prevesti u jezik formalne logike, te ćemo na taj način dobiti sljedeće četiri klauze:

$$k1 = (A \vee (\neg B)) \Rightarrow (C \text{ exor } (\neg D))$$

$$k2 = C \Leftrightarrow AB$$

$$k3 = \neg B \Rightarrow ((\neg A) \text{ exor } (\neg C))$$

$$k4 = D \Rightarrow (\neg B)C$$

Ove klauze šaljemo u solver i na taj način se od njih kreira konjunktivna normalna forma.

```
1  import z3
2
3  a = z3.Bool('a')
4  b = z3.Bool('b')
5  c = z3.Bool('c')
6  d = z3.Bool('d')
7
8  k1 = z3.Implies(z3.Or(a, z3.Not(b)), z3.Xor(c, z3.Not(d)))
9  k2 = (c == z3.And(a, b))
10 k3 = z3.Implies(z3.Not(b), z3.Xor(z3.Not(a), z3.Not(c)))
11 k4 = z3.Implies(d, (z3.And(z3.Not(b), c)))
12
13 s = z3.Solver()
14
15 s.add(k1)
16 s.add(k2)
17 s.add(k3)
18 s.add(k4)
19
20 provjera = s.check()
21 print("Provjera: ", provjera)
22 # pronalaženje i ispisivanje modela za koje je kreirana formula tačna
23 while s.check() == z3.sat:
24     model = s.model()
25     print("Model:", model)
26     lista = []
27     for var in model:
28         c = var()
29         lista.append(c != model[var])
30     s.add(z3.Or(lista))
31
```

Figure 14: Primjer 2 - realizacija, prvi dio

```

31
32 binarni = ['0000', '0001', '0010', '0011', '0100', '0101', '0110', '0111',
33           '1000', '1001', '1010', '1011', '1100', '1101', '1110', '1111']
34
35 print("TABLICA ISTINE:")
36 print()
37 print("-----")
38 print("| a | b | c | d | IZRAZ |")
39 print("-----")
40
41 for i in range(16):
42     A = bool(int(binarni[i][0]))
43     B = bool(int(binarni[i][1]))
44     C = bool(int(binarni[i][2]))
45     D = bool(int(binarni[i][3]))
46
47     k1 = z3.Implies(z3.Or(A, z3.Not(B)), z3.Xor(C, z3.Not(D)))
48     k2 = (C == z3.And(A, B))
49     k3 = z3.Implies(z3.Not(B), z3.Xor(z3.Not(A), z3.Not(C)))
50     k4 = z3.Implies(D, z3.And(z3.Not(B), C))
51
52     s1 = z3.Solver()
53     s1.add(k1)
54     s1.add(k2)
55     s1.add(k3)
56     s1.add(k4)
57
58     print("|", "{:<5}".format(A), "|", "{:<5}".format(B), "|", "{:<5}".format(C),
59           "|", "{:<5}".format(D), "|", end = " ")
60     if s1.check() == z3.sat:
61         print(" 1 |")
62     else:
63         print(" 0 |")
64     print("-----")
65

```

Figure 15: Primjer 2 - realizacija, drugi dio

Prethodne dvije slike prikazuju implementaciju traženog logičkog izraza, dok je tabela istine prikazana na slici ispod. Vidimo da je kreirani izraz zadovoljavajući, međutim, u tablici istine vidimo da on dostiže vrijednost "True" samo za dva modela ulaznih literala. Ovo nam sada daje jasnu predstavu o razlici između pojmova tautologija i "SAT". Zaključujemo da je svaka tautologija ujedno i zadovoljavajući izraz, ali obrnuto ne vrijedi, to jest, nije svaki zadovoljavajući izraz tautologija.

Provjera: sat
Model: [b = True, a = False, c = False, d = False]
Model: [b = False, a = True, c = False, d = False]
TABLICA ISTINE:

a	b	c	d	IZRAZ
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Figure 16: Primjer 2 - ispis

9.3 Primjer 3

Kandidat pred lokalne izbore obećava sljedeće:
„Ili ću vam napraviti školu, ili ću vam, ukoliko dobijemo krediti, dati trinaestu penziju. Kredit nećemo dobiti samo ako vam napravim školu. Ustvari, ako dobijemo kredit, dobit ćete oboje, i školu, i trinaestu penziju.“ Ispitati da li je ovo obećanje konzistentno ili kontradiktorno.

Prevedemo li ovo obećanje u jezik formalne logike, dobit ćemo sljedeće tri klauze:

$k1 = A \text{ exor } (B \Rightarrow C)$

$k2 = B \vee A$

$k3 = B \Rightarrow AC$

Ove tri klauze šaljemo solveru koji od njih kreira konjunktivnu normalnu formu, te za na taj način kreiran logički izraz ispisujemo tablicu istine. Vidimo da ni za jedan model ulaznih literala logički izraz ne dostiže vrijednost "True" što se poklapa sa činjenicom da je on "unsat" koju nam daje poziv funkcije check() nad kreiranim solverom.

```
1  import z3
2
3  x = z3.Bool('x')
4  y = z3.Bool('y')
5  z = z3.Bool('z')
6
7  k1 = z3.Xor(x, z3.Implies(y, z))
8  k2 = z3.Implies(z3.Not(y), x)
9  k3 = z3.Implies(y, z3.And(x, z))
10
11 s = z3.Solver()
12
13 s.add(k1)
14 s.add(k2)
15 s.add(k3)
16
17 provjera = s.check()
18 print("Provjera: ", provjera)
19
20 # pokušaj pronalaska modela za kojeg je izraz zadovoljavajući
21 try:
22     model = s.model()
23     print("Model: ", model)
24 except:
25     print("Greška prilikom dohvaćanja modela!")
26
27 # sada imamo tri literala pa nam je potrebno samo 2^3 = 8 binarnih brojeva
28 # ove brojeve sada zapisujemo na tri bita
29 binarni = ['000', '001', '010', '011', '100', '101', '110', '111',]
30
31 print("TABLICA ISTINE:")
32 print()
33 print("-----")
34 print("| x | y | z | IZRAZ |")
35 print("-----")
36
37 for i in range(8):
38     x = bool(int(binarni[i][0]))
39     y = bool(int(binarni[i][1]))
40     z = bool(int(binarni[i][2]))
41
42     k1 = z3.Xor(x, z3.Implies(y, z))
43     k2 = z3.Implies(z3.Not(y), x)
44     k3 = z3.Implies(y, z3.And(x, z))
45
46     s1 = z3.Solver()
47     s1.add(k1)
48     s1.add(k2)
49     s1.add(k3)
50
51     print("|", "{:<5}".format(x), "|", "{:<5}".format(y), "|", "{:<5}".format(z), "|", end = " ")
52     if s1.check() == z3.sat:
53         print(" 1 |")
54     else:
55         print(" 0 |")
56     print("-----")
57
```

Figure 17: Primjer 3 - realizacija

Provjera: unsat				
Greška prilikom dohvaćanja modela!				
TABLICA ISTINE:				

x	y	z	IZRAZ	

0	0	0	0	

0	0	1	0	

0	1	0	0	

0	1	1	0	

1	0	0	0	

1	0	1	0	

1	1	0	0	

1	1	1	0	

Figure 18: Primjer 3 - ispis

Važno je napomenuti da solveri kreirani u prethodna tri primjera nisu klasični SAT solveri nego njihovo poopćenje. Klasični SAT solveri na ulazu primaju samo klauze sastavljene korištenjem literala i osnovnih logičkih operacija AND, OR i NOT od kojih se dalje, jednostavnim povezivanjem ovih klauza u konjunkciju, kreira konjunktivna normalna forma. S druge strane, solverima iz z3 biblioteke možemo slati puno širi opseg klauza, a zatim se ovaj solver oslanja na klasični SAT solver i pri tome obavi za nas jedan dio posla vezanog za kreiranje konjunktivne normalne forme.

Istaknimo i to da SAT solveri iz dana u dan doživljavaju ogroman napredak te da prethodna tri primjera prikazuju njihovu upotrebu na nekom najosnovnijem nivou. Tokom posljednjih godina na SAT takmičenjima brojni problemi koji su prije desetak godina izgledali nerješivi sada su postali stvar rutine. Pored mnogobrojnih novih algoritama i heuristike, ispostavilo se da su rafinirane tehnike implementacije bile ključne za uspjeh. Također danas su mnogi moderni SAT solveri otvorenog koda i dostupni javnosti, što omogućava široku upotrebu i mogućnost prilagodbe po potrebi.

10 Zaključak

Revolucionarna Cook-Levin teorema dala je SAT problemu posebno mjesto u informatičkoj nauci, gdje interes za njega i dalje živi. Nemogućnost potpune algoritamske rješivosti zbog njegove NP kompletnosti dovela je do brojnih strategija i algoritama kojima se dolazi do praktične aproksimacije rješenja za svakodnevnu primjenu. Ova rješenja se sprovode unutar jednog od raznih vrsta SAT solvera, programa koji pokušavaju naći konfiguraciju literala u logičkoj Booleovoj formuli koja tu formulu zadovoljava i time rješava problem. Tokom posljednje dvije decenije, tehnike rješavanja SAT-a promijenile su način na koji rješavamo teške računske probleme. SAT solveri su tiho utjecali na modernu tehnologiju. Koriste se u računarskoj biologiji, za planiranje, za verifikaciju modernog hardvera, operativnih sistema, softvera, pa čak i matematičkih iskaza. SAT solveri napreduju i mada teorija računljivosti ne polaže nade da će se ikada uspjeti doći do idealnog SAT solvera, nije dokazano da ne postoji algoritam rješavanja NP kompletnosti, što ostavlja tračak naivne nade za budućnost.

11 Izvještaj

Svi članovi tima su se potrudili da daju svoj maksimalni doprinos pri izradi ovog seminarskog rada. Svi su odgovorno pristupili svom dijelu zadatka, a nije manjkalo ni timskog rada i međusobnog pomaganja. Tako su uvod, historijski razvoj i zaključak obradile Fadila Begić i Nedžmija Karić, vrste SAT solvera Faris Aljić, sekcije 4, 5, 6, 7 i 8 Zerina Pandža, dok je Enis Adilović svoj doprinos dao na način da je sve to pokušao praktično primijeniti kroz nekoliko primjera. Naravno, ovo nije fiksna raspodjela zadataka iz razloga što su svi bili otvoreni za saradnju i rado uvažavali savjete drugih te na taj način zajedničkim snagama priveli ovaj projekat kraju.

12 Izvori

Hecher, Markus, Fichte, Johannes, Le Berre, Daniel and Szeider, Stefan. 2023. "The Silent (R)evolution of SAT."

Up and Atom: "NP-COMPLETENESS - The Secret Link Between Thousands of Unsolved Math Problems"

Renato Lui Geh, "A POLYNOMIAL-TIME REDUCTION OF THE 3-SAT TO THE QUADRATIC CONGRUENCE AND OTHER RELATED PROBLEMS"

BorealisAI tutorial9 SAT solvers part1

BorealisAI tutorial10 SAT solvers part2

Solving SAT solvers with z3

Modern SAT solvers by Martin Horenovsky

SAT solvers Wikipedia

Boolean satisfiability problem Wikipedia

The International SAT Solver Competitions

An Analysis and Comparison of Satisfiability Solving Techniques

Lecture Notes on SAT Solvers and DPLL

Modern SAT Solvers